NAME: TANKISO MASOEBE

COMPUTER SECURITY WEEK 12

1. HANDS-ON HARDENING LAB REPORT

Title: Basic System Hardening Using Ubuntu Linux

1. Introduction

System hardening is the process of making a computer system more secure by removing unnecessary services, installing updates, controlling user access, and configuring a firewall. In this practical, I used an Ubuntu Linux virtual machine to perform basic system hardening.

2. Updating the System

First, I started my Ubuntu virtual machine and opened the Terminal. I entered the following commands to update the system:

```
sudo apt update
sudo apt upgrade -y
```

The first command checked for available updates, while the second command installed the updates.

Result: The system was updated to fix bugs, improve performance, and protect against security vulnerabilities.

3. Disabling Two Non-Essential Services

I checked the running services using the following command:

```
systemctl --type=service --state=running
```

I selected Bluetooth and Avahi network discovery as services that were not required for my lab environment.

Service 1: Bluetooth

I used the following command to stop and disable Bluetooth:

sudo systemctl disable --now bluetooth

Service 2: Avahi

Avahi allows devices to discover services on a local network. Since it was not needed for my practical, I disabled it using:

sudo systemctl disable --now avahi-daemon

I checked the status of the services to confirm whether they had been disabled.

Result: Disabling unnecessary services reduced background activity and helped reduce possible security risks.

4. Creating a Standard User Account

I created a new standard user account called student1 by entering:

sudo adduser student1

I followed the instructions to create a password and complete the account details.

I then switched to the new account using:

su - student1

To confirm that I was using the new account, I entered:

whoami

The output was:

student1

I used this account for normal activities instead of using the administrator account. This helps prevent accidental system changes and reduces the risk of unauthorized access.

Result: A standard user account was created for everyday use, while administrator privileges were reserved for tasks that required them.

5. Configuring the Firewall

I used Ubuntu's built-in firewall management tool, UFW, to control incoming network connections.

First, I installed UFW if it was not already installed:

```
sudo apt install ufw -y
```

Next, I configured the firewall to block all incoming connections by default and allow outgoing connections:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

I allowed SSH connections, which use port 22 by default:

```
sudo ufw allow OpenSSH
```

Finally, I enabled the firewall:

```
sudo ufw enable
```

I checked the firewall configuration using:

```
sudo ufw status verbose
```

The firewall was expected to block incoming connections by default while allowing SSH connections.

Result: The firewall restricted incoming network traffic and allowed only the specified SSH service. This helped protect the system from unwanted network connections.

Note: SSH must be installed and running for remote SSH access to work. If I was connected to the virtual machine remotely, I would allow SSH before enabling the firewall to avoid losing access.

6. Conclusion

In conclusion, I performed basic system hardening on an Ubuntu Linux virtual machine. I updated the operating system, disabled two non-essential services, created a standard user account, and configured the firewall to block incoming connections except for SSH.

These activities helped improve system security by reducing unnecessary services, limiting administrator access, and controlling network traffic. System hardening is important because it reduces security risks and helps protect computer systems from unauthorized access.

7. Evidence and Screenshots

The following screenshots should be included as evidence of the practical:

1. Screenshot showing the system update commands.
2. Screenshot showing the commands used to disable Bluetooth and Avahi.
3. Screenshot showing the whoami command and the student1 username.
4. Screenshot showing the output of sudo ufw status verbose.

2. Comparative Analysis: SELinux vs. AppArmor

1. Introduction

SELinux and AppArmor are Linux security systems that help protect computers from unauthorized access. They use Mandatory Access Control (MAC), which means that security rules control what users and programs are allowed to do. Both systems improve security by limiting the actions of applications, even when an application is compromised.

2. Design Philosophy

SELinux (Security-Enhanced Linux) uses Type Enforcement to control access to system resources. It assigns security labels to processes, files, and other resources. Rules determine which processes can access particular resources. For example, a web server may be allowed to read website files but prevented from accessing sensitive system files. SELinux provides detailed security controls and is commonly used in Red Hat Enterprise Linux and Fedora.

AppArmor (Application Armor) mainly uses path-based security profiles. It controls application access using file paths and defined permissions. For example, a profile can allow a web server to read files in its website directory while preventing access to other directories. AppArmor is commonly used in Ubuntu and SUSE Linux distributions.

3. Complexity of Management

SELinux is generally more complex to configure and manage because administrators must understand security labels, policies, contexts, and access rules. Troubleshooting denied access may require checking audit logs and correcting file labels or policies. However, its detailed controls make it suitable for systems that require strong security.

AppArmor is generally easier to learn and manage. Administrators can create profiles that specify which files and resources an application may access. It also provides tools to help generate and update profiles. This makes AppArmor a practical choice for administrators who want to improve security without managing highly complex policies.

4. Typical Use Cases

SELinux is often used in enterprise environments, government systems, and servers that require strict security controls. It is suitable for systems that handle sensitive information, such as database servers and enterprise applications.

AppArmor is commonly used on Ubuntu-based systems, personal computers, and web servers. It is useful when administrators need application-level security that is relatively simple to configure and maintain.

5. Recommendation

For a small web server running Ubuntu Linux, I would recommend AppArmor because it is easier to manage and provides useful protection for applications such as Apache or Nginx. It can restrict the files and resources that a web server can access, reducing the damage caused by a possible attack.

However, for a large enterprise database server that requires detailed security policies, I would consider SELinux because it provides fine-grained access control through security labels and policy rules.

6. Conclusion

In conclusion, SELinux and AppArmor both improve Linux security through Mandatory Access Control. SELinux offers detailed, label-based security but is more complex to manage, while AppArmor uses path-based profiles and is generally easier to configure. The best choice depends on the operating system, security requirements, and the administrator's experience.